

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#8

JavaScript & and Steps to React

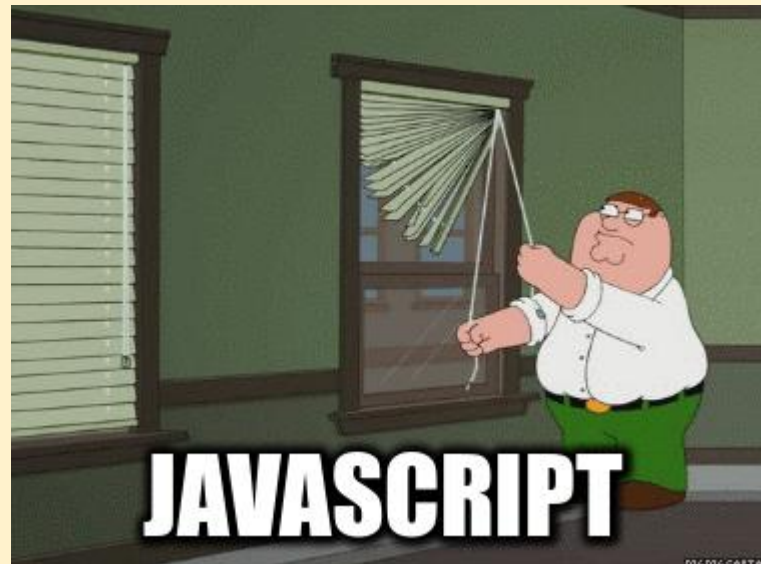
Lecture Outline

Dr. Mohamed Sami Rakha

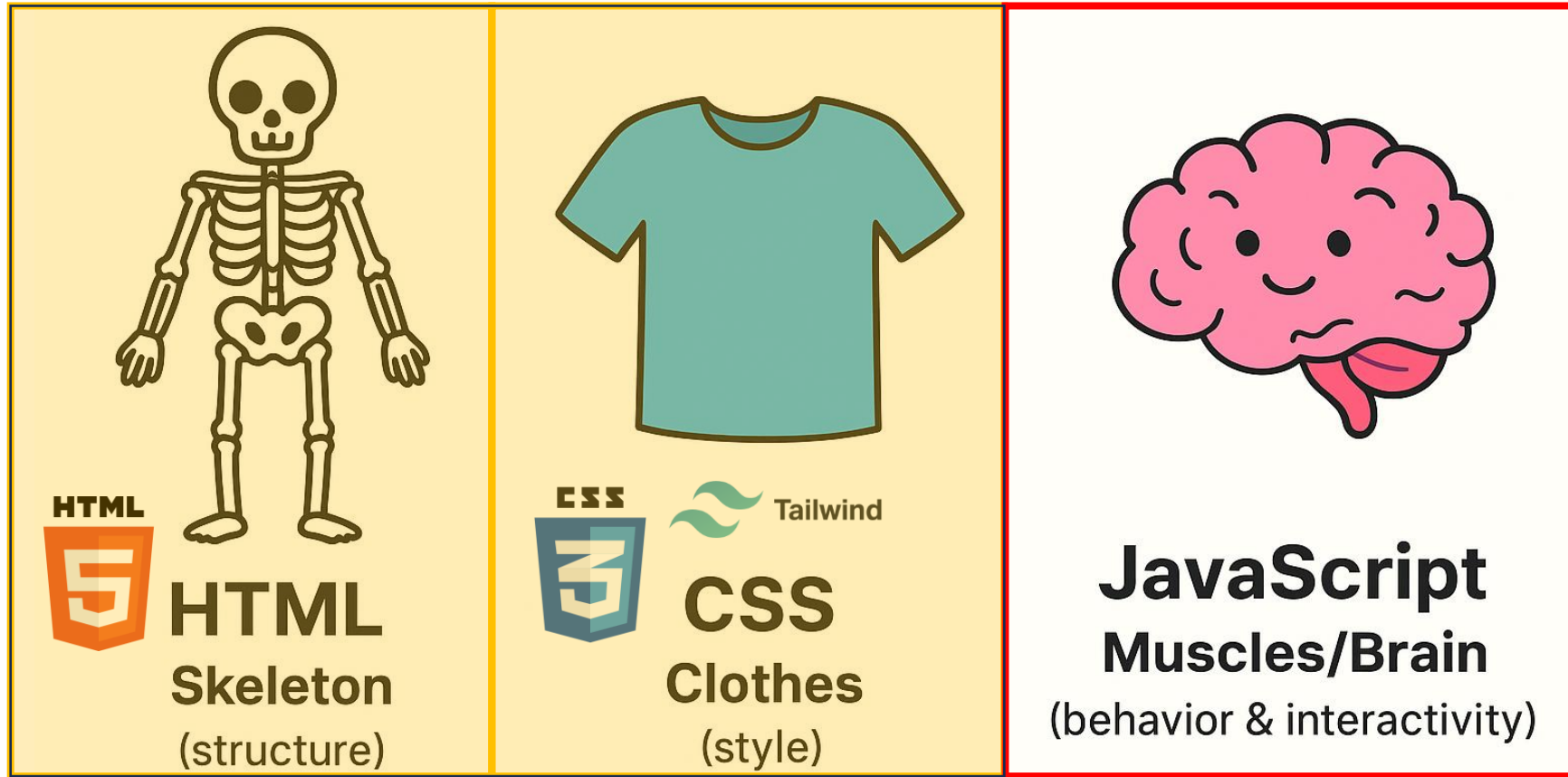
Cont JavaScript

Steps to react JavaScript

JavaScript



(HTML, CSS, JavaScript)



Rakha



DOM The interface that connects
HTML (structure), CSS (style),
and JavaScript (behavior)

Building Blocks of Scripts

Dr. Mohamed Sami Rakha

- Variables
- Comparison operators
- if/else statements
- Loops
- Functions
- Scope

Define Constant

In JavaScript, you can define a constant using the **const** keyword. A constant is a variable whose value cannot be reassigned once it's set. Here's the basic syntax:

```
const PI = 3.14159;
```

You must assign a value when you declare a **const**. This will throw an error:

```
const x; // ❌ Error: Missing initializer
```

You cannot reassign a constant:

```
const name = "Mo";  
name = "Rakha"; // ❌ Error: Assignment to constant variable
```

Const is hoisted with the same rules as "let"

Define Constant

The value itself is not always **immutable** (unalterable). For example, objects or arrays declared with `const` can still have their contents changed:

```
const person = { name: "Mo" };  
person.name = "Rakha"; //  This is allowed
```

Use **const** when you don't intend to reassign the variable—it's a best practice for safer, more predictable code.

JavaScript Strings

What is a String? A string is a sequence of characters used to store text. Can be written with "double", 'single', or `backticks`.

Declaring Strings

```
let a = "Hello";  
let b = 'World';  
let c = `Hello ${b}`; // template literal
```

Key Properties

```
text.length // number of characters  
text[index] // access character
```

Immutability

Strings cannot be changed, only replaced.

```
let s = "Hello";  
s[0] = "J"; // ✗ does nothing  
s = "Jello"; // ✓ correct way
```

This is why most string methods return new strings.

Concatenation

```
let full = "UI" + " Development";
```

```
let msg = "Hello";  
msg += " World";
```


JavaScript Strings

What is a String? A string is a sequence of characters used to store text. Can be written with "double", 'single', or `backticks`.

Template Literals

Using backticks (`):

```
let name = "Ali";  
let greeting = `Hello, ${name}!`;
```

```
let user = "Sara";  
let score = 95;  
  
console.log(`User ${user} scored ${score} points`);
```

Common String Methods

Methods that *return new strings*:

- .toUpperCase()
- .toLowerCase()
- .trim()
- .slice()
- .replace()
- .includes()
- .split()

Frequently used in DOM interactions.

Case-insensitive search

```
Dr. "hello".toUpperCase() // "HELLO"  
"HELLO".toLowerCase() // "hello"
```

Removes spaces at start and end.

```
"  hello  ".trim() // "hello"
```

Extracts part of a string

```
let x = "JavaScript";  
x.slice(0, 4); // "Java"  
x.slice(4); // "Script"
```

Checks if a string contains another string.

```
"UI Development".includes("UI") // true
```

Replaces the first matched substring.

```
"Hello World".replace("World", "Students");  
// "Hello Students"
```

Converts a string into an array

```
"Ali, Sara, Omar".split(", ");  
// ["Ali", "Sara", "Omar"]
```

Event Handlers

Event Handlers

An **event** is an action that can be detected with JavaScript and used to trigger scripts.

Common Browser Events

Mouse Events

- click
- dblclick
- mouseover
- mouseout
- mousedown
- mouseup

Keyboard Events

- Keydown
- Keyup

Form Events

- submit
- change
- input
- focus
- blur

Window Events

- load
- resize
- scroll

Event Handlers

An **event** is an action that can be detected with JavaScript and used to trigger scripts.

Events are identified by **event handlers**. Examples:

- `onload` When the page loads
- `onclick` When the mouse clicks an object
- `onmouseover` When the pointer is moved over an element
- `onerror` When an error occurs when the document or a resource loads

Event Handlers

Event handlers can be applied to items in pages in three ways:

1- Inline HTML Event Handler

- As an HTML attribute:

```
<body onclick="myFunction();" >  
/* myFunction runs when the user clicks anything  
within 'body' */
```

html

```
<button onclick="alert('Clicked!')">Click Me</button>
```

Event Handlers

Event handlers can be applied to items in pages in three ways:

2- DOM Property Method with Function Attached

```
<button id="btn">Click</button>

<script>
document.getElementById("btn").onclick = function () {
    alert("Button clicked!");
};
</script>
```

- As a method attached to the element:

```
window.onclick = myFunction;
/* myFunction will run when the user clicks anything
within the browser window */
```

Event Handlers

Event handlers can be applied to items in pages in three ways:

3- `addEventListener()` --- BEST PRACTICE

- Using `addEventListener()`:

```
window.addEventListener("click", myFunction);
```

Notice that we omit the preceding “on” from the event handler with this syntax.

```
<button id="btn">Click</button>

<script>
document.getElementById("btn")
    .addEventListener("click", function () {
        alert("Clicked!");
    });
</script>
```

- ✓ Supports multiple event handlers
- ✓ Cleaner separation of HTML & JavaScript
- ✓ Supports advanced features (event options, removal)

Event Options

addEventListener() allows you to add optional settings such as:

Once, run event 1 time only

```
button.addEventListener("click", () => {  
  console.log("This will run only once!");  
}, { once: true });
```

passive — improve scrolling performance

Useful for touch and scroll events.

```
window.addEventListener("scroll", (e) => {  
  console.log("Scrolling...");  
}, { passive: true });
```

By default, some events (like `scroll` or `touchmove`) can block the browser while running JavaScript.

`passive: true` tells the browser:

“I promise I won’t call `preventDefault()` in this handler.”
→ The browser can scroll smoothly without waiting for JS.

Event Options

You can remove an event listener, but only if you use `addEventListener()`:

```
function sayHi() {  
    console.log("Hello");  
}  
  
button.addEventListener("click", sayHi);  
  
// Remove it later:  
button.removeEventListener("click", sayHi);
```

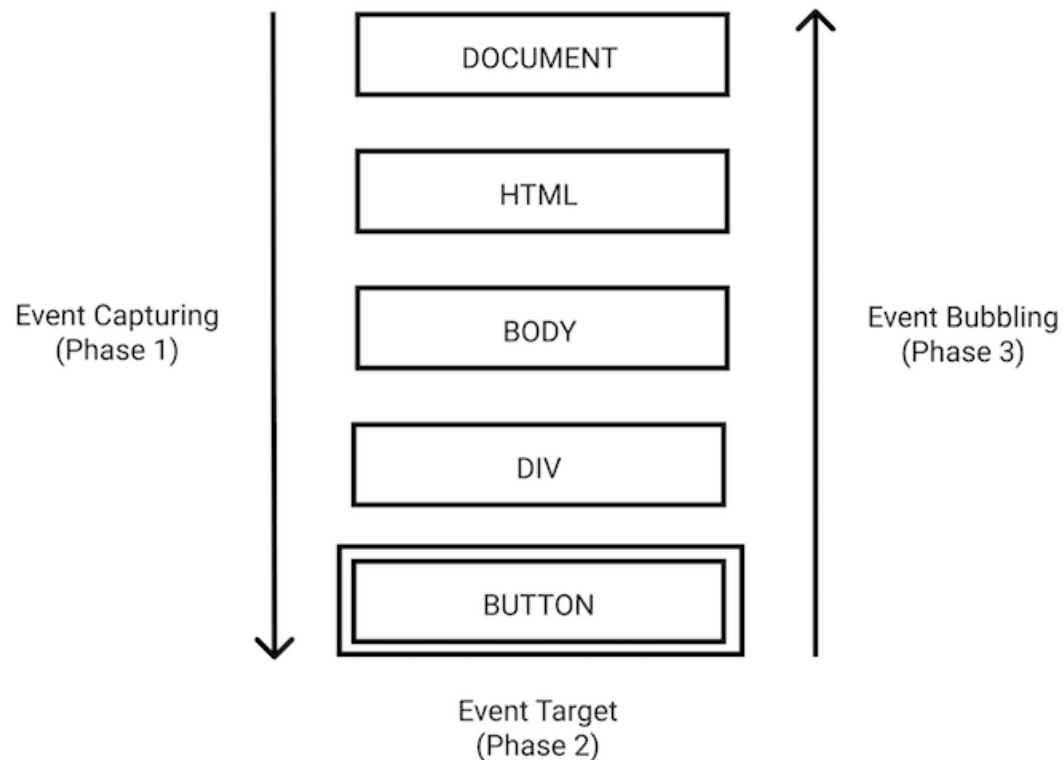
This is NOT possible with `onclick`, because you cannot remove inline handlers in the same way.

Event Delegation in JavaScript

What is Capturing?

When an event happens in the DOM, it doesn't just bubble up. It first travels from the top of the DOM down to the target element. This first phase is called **capturing**.

Dr. Mohamed Sami Rakha



When a user clicks an element:

1. **Capturing phase:** the event travels from **document → html → body → parent → target**
2. **Target phase:** the event reaches the element that triggered it
3. **Bubbling phase:** the event **bubbles back up** to the ancestors

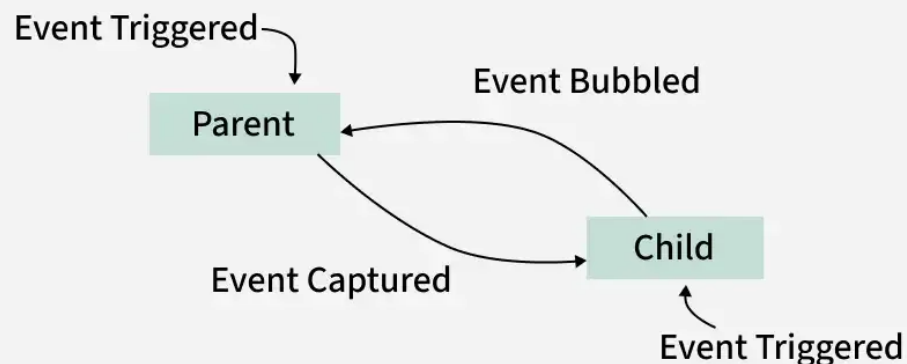
Event Delegation in JavaScript

What is Event Delegation?

- A technique to handle events efficiently by attaching a single event listener to a parent element instead of multiple children.
- Works because of event **bubbling**: events “bubble up” from child elements to parent elements.
- Common use case: lists, tables, dynamic content where elements are added or removed dynamically.

Dr. Mohamed Sami Rakha

Event Bubbling and Capturing

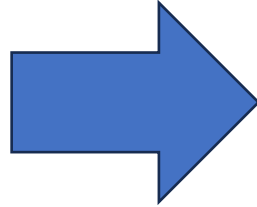


Why Use Event Delegation?

- Fewer event listeners → better performance
- Works for dynamically added elements
- Keeps code clean and maintainable

Event Delegation in JavaScript: Example

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```



Instead of attaching a click event to each :

```
const items = document.querySelectorAll("#myList li");
items.forEach(item => {
  item.addEventListener("click", () => console.log("Clicked!"));
});
```



Let's Use Event Delegation

```
document.getElementById("myList").addEventListener("click", function(e) {
  if(e.target.tagName === "LI") {
    console.log("Clicked:", e.target.textContent);
  }
});
```

How Event Delegation Works

1- Event Bubbling

When a child element is clicked:

- Event triggers on the **child** (target)
- Event bubbles **up the DOM** to parent elements
- Parent can **handle all child events** in one listener

2. Using event.target

event.target tells which element triggered the event

Allows the parent to filter events for specific children

```
parent.addEventListener("click", function(e) {  
    if(e.target.matches("button.remove")) {  
        console.log("Remove button clicked:", e.target);  
    }  
});
```

How Event Delegation Works

Dynamic Elements Example

Dr. Mohamed Sami Rakha

```
<ul id="tasks"></ul>
<button id="addTask">Add Task</button>

<script>
const tasks = document.getElementById("tasks");
document.getElementById("addTask").addEventListener("click", () => {
  const li = document.createElement("li");
  li.textContent = "New Task";
  tasks.appendChild(li);
});

// Handle clicks using delegation
tasks.addEventListener("click", function(e) {
  if(e.target.tagName === "LI") {
    e.target.style.textDecoration = "line-through";
  }
});
</script>
```

Newly added `<li>` elements automatically work without adding new listeners.

Event Propagation Phase

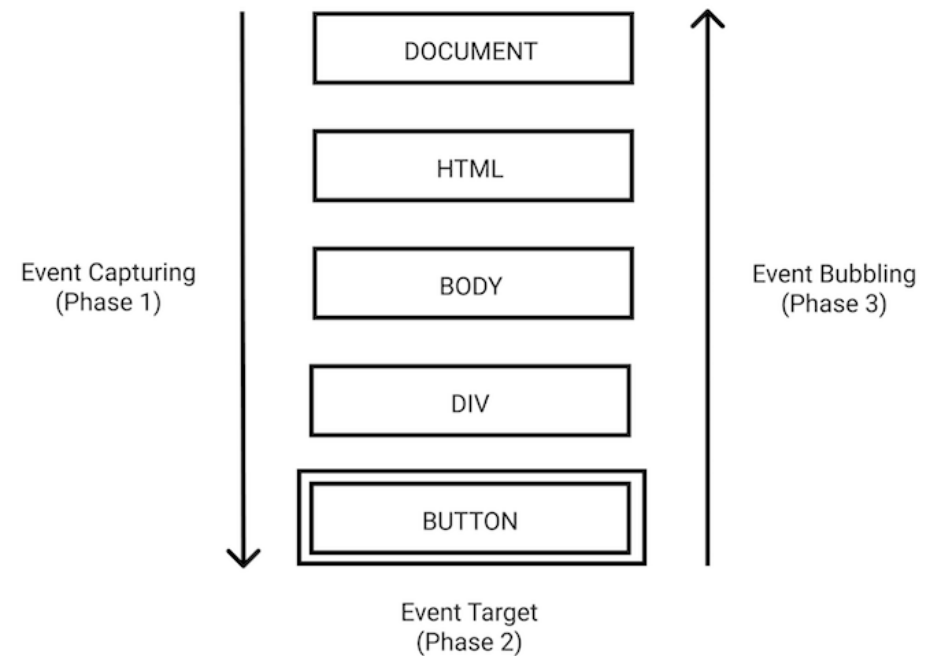
Example on Event Propagation

```
<div id="parent">
  <button id="child">Click Me</button>
</div>

<script>
document.getElementById("parent").addEventListener("click", () => {
  console.log("Parent clicked!");
});

document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked!");
});
</script>
```

What happens when you click the **button**?



Event Propagation Phase

Example on Event Propagation

Dr. Mohamed Sami Rakha

```
<div id="parent">
  <button id="child">Click Me</button>
</div>

<script>
document.getElementById("parent").addEventListener("click", () => {
  console.log("Parent clicked!");
});

document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked!");
});
</script>
```

What happens when you click the button?

- **Target phase** → event reaches the button → logs "Child clicked!"
- **Bubbling phase** → event bubbles up to div#parent → logs "Parent clicked!"

Event Delegation + Capturing

- Normally, Event Delegation relies on bubbling.
- If you set `capture: true` (Default is false), the parent catches the event before the child.

If you set `capture: true`, the parent catches the event **before the child**.

```
document.getElementById("parent")
  .addEventListener("click", function(e) {
    console.log("Parent captured first");
  }, { capture: true });

document.getElementById("child")
  .addEventListener("click", function(e) {
    console.log("Child clicked");
  });
```

Clicking the child → Output:

```
Parent captured first
Child clicked
```

- **Bubbling** is most common, use it for normal event delegation
- **Capturing** is useful when you want to **intercept events early**
- Can combine capturing and bubbling to handle events at **different stages**

Change the Styles of Elements with Events

Modify Styles of Elements - Example

Hover effect via JavaScript

```
<p id="text">Hover over me!</p>

<script>
  const text = document.getElementById("text");

  text.addEventListener("mouseover", () => {
    text.style.backgroundColor = "yellow"; // highlight
  });

  text.addEventListener("mouseout", () => {
    text.style.backgroundColor = ""; // remove
    highlight
  });
</script>
```

Dr. Mohamed Sami Rakha

On Mouseover

Hover over me!

On Mouseover

Hover over me!

Modify Styles of Elements - Example

Click to toggle highlight

```
<p id="clickText">Click me to highlight!</p>

<script>
  const clickText = document.getElementById("clickText");

  clickText.addEventListener("click", () => {
    if (clickText.style.backgroundColor === "green") {
      clickText.style.backgroundColor = "";
    } else {
      clickText.style.backgroundColor = "green";
    }
  });
</script>
```

Dr. Mohamed Sami Rakha

On Click

Click me to highlight!

On Click

Click me to highlight!

Modify Styles of Elements - Example

Show/Hide Content

```
<button id="toggleBtn">Click Here!</button>
<p id="content">This is some text to show or hide.</p>

<script>
  const btn = document.getElementById("toggleBtn");
  const content = document.getElementById("content");

  btn.addEventListener("click", () => {
    if (content.style.display === "none") {
      content.style.display = "block"; // show
    } else {
      content.style.display = "none"; // hide
    }
  });
</script>
```

Dr. Mohamed Sami Rakha

On Click

Click Here!

This is some text to show or hide.

On Click

Click Here!

Modify Styles of Elements - Example

These two patterns (**highlighting + toggling visibility**) are the **foundation for building interactive UIs** — menus, modals, tooltips, and more.

highlighting



The diagram shows two examples of highlighting within a large orange-bordered box. The top example shows a yellow box labeled 'Hover over me!' with the label 'On Mouseover' above it, and a white box labeled 'Click me to highlight!' with the label 'On Click' above it. The bottom example shows a white box labeled 'Hover over me!' with the label 'On Mouseover' above it, and a green box labeled 'Click me to highlight!' with the label 'On Click' above it.

toggling visibility



The diagram shows two examples of toggling visibility within a large orange-bordered box. The top example shows a button labeled 'Click Here!' with the label 'On Click' above it, followed by the text 'This is some text to show or hide.' The bottom example shows a button labeled 'Click Here!' with the label 'On Click' above it, and the text area below it is empty.

JavaScript Libraries

A **JavaScript library** is a collection of prewritten functions and methods that you can use in your scripts to accomplish common tasks or simplify complex ones.

- Some are large frameworks for building complex applications. Like **“React”**
- Some are targeted to specific tasks, such as forms or math.
- Popular library is **jQuery**.

Forms & Input Handling

Forms & Input Handling in JavaScript

1. Selecting Form Elements

```
<form id="myForm">
  <input type="text" id="name" placeholder="Enter your name">
  <button type="submit">Submit</button>
</form>
```

```
const form = document.getElementById("myForm");
const nameInput = document.getElementById("name");
```

Use `getElementById`, `querySelector`, or `querySelectorAll`

2. Reading Input Values

```
const value = nameInput.value;
console.log("User entered:", value);
```

`.value` retrieves what the user typed

3. Responding to Input Events

```
nameInput.addEventListener("input", () => {
  console.log("Typing:", nameInput.value);
});
```

Input: triggers on every change while typing

Handling Form Submission

1. Listening for Submit

```
form.addEventListener("submit", function(e) {  
  e.preventDefault(); // Prevent page reload  
  console.log("Form submitted! Name:", nameInput.value);  
});
```

submit event fires when the form is submitted
event.preventDefault() stops the default form behavior (page reload)

2. Validating Input

```
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
  if(nameInput.value.trim() === "") {  
    alert("Please enter your name!");  
  } else {  
    console.log("Form is valid!");  
  }  
});
```

Simple client-side validation
before sending data

3. Dynamic Form Handling

```
form.addEventListener("input", (e) => {  
  if(e.target.tagName === "INPUT") {  
    console.log("Changed:", e.target.value);  
  }  
});
```

Forms can have dynamic input fields
Use event delegation for inputs added dynamically

Modules

Named Exports

What Are Modules?

A module is a JavaScript file that **exports** code (variables, functions, classes) to be reused in other files.

- Helps organize code and avoid polluting the global scope.
- Prepares students for React component structure (each component is a module).

Exporting from a Module

```
// file: mathUtils.js
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}
```

export makes code available to other files
Can export variables, functions, classes

Importing Modules

```
// file: main.js
import { PI, add } from './mathUtils.js';

console.log("PI =", PI);
console.log("2 + 3 =", add(2, 3));
```

Use **{ }** to select specific exports
Can rename with **as**:

```
import { add as sum } from './mathUtils.js';
console.log(sum(4,5)); // 9
```

Default Export

A module can have **only one default export**
Designed for the “main thing” a module provides
Imported without { } and you can give it any name

```
// logger.js
export default function log(message) {
  console.log(message);
}

// main.js
import myLogger from './logger.js';
myLogger("Hello"); // Works fine
```

- Simplifies imports for modules that export one main thing
- Flexible naming when importing (myLogger, log, etc.)

Why Use Default Export?

1. When your file **focuses on a single functionality**

- e.g., a **React component**, a class, a main function

2. Makes the import **shorter and easier**

3. Allows **flexible naming** when importing

Default Export / Import

Use Case	Export Type
One main thing per file (e.g., React component, a class)	Default export
Multiple utilities in one file (e.g., constants, functions)	Named exports

Example in React Context:

```
// Button.js
export default function Button() { return <button>Click</button>; }

// utils.js
export function formatDate() {...}
export function calculateTotal() {...}
```

- React components usually use **default export**
- Utility files often use **named exports**

Error Handling

try/catch Blocks

Use try/catch to **catch runtime errors** and handle them safely

```
try {  
  const result = riskyOperation();  
  console.log("Result:", result);  
} catch (error) {  
  console.error("An error occurred:", error.message);  
}
```

- try { ... } → code that might throw an error
- catch (error) { ... } → code to handle the error
- error.message → human-readable description of the error

finally block always runs, whether error occurs or not

```
try {  
  console.log("Trying risky code");  
} catch(e) {  
  console.error(e);  
} finally {  
  console.log("Cleanup code always runs");  
}
```

try/catch allows graceful error handling

```
try {  
  JSON.parse("invalid json");  
} catch (e) {  
  console.error("Failed to parse JSON:", e);  
}
```

Useful for debugging without stopping the program

Console Methods

- console.log(value) → general info
- console.warn(value) → warning messages
- console.error(value) → error messages
- console.table(arrayOrObject) → display in table format

Browser APIs: Storage & Fetch

What Are Browser APIs?

JavaScript interfaces are provided by the browser. Allow interaction with browser features:

Storage: localStorage, sessionStorage

Networking: fetch()

DOM, Geolocation, Notifications, etc.

Feature	localStorage	sessionStorage
Lifetime	Permanent until cleared	Only for current tab session
Scope	All tabs from the same origin	Single tab only
Storage Type	Key-value strings	Key-value strings

What Are Browser APIs?

Using localStorage

```
// Store
localStorage.setItem("username", "Ali");

// Retrieve
const user = localStorage.getItem("username");
console.log(user); // Ali

// Remove
localStorage.removeItem("username");

// Clear all
localStorage.clear();
```

Using sessionStorage

```
sessionStorage.setItem("sessionID", "12345");
console.log(sessionStorage.getItem("sessionID"));
```

Use **localStorage** for long-term data, **sessionStorage** for temporary session data.

Fetch API for API Calls

What is fetch()?

- Modern way to make **HTTP requests** in the browser
- Returns a **Promise**
- Typically used with **JSON APIs**

Dr. Mohamed Sami Rakha

GET Request

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error:", error));
```

`fetch()` is **Promise-based** → asynchronous

Use `.then()` or `async/await` for handling responses

`.then()` → executes when the Promise **resolves**

`.catch()` → executes when the Promise **rejects**

POST Request

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hello", body: "World" })
})
  .then(res => res.json())
  .then(data => console.log("Created:", data))
  .catch(err => console.error(err));
```

JS continues running the next lines **without waiting**
When the Promise **resolves**, the `.then()` callback runs

Dr. Mohamed Sami Rakha

Thank You...